Forward and Inverse Kinematics of the UR5 Robot with Task-Space Trajectory Tracking

Student Name: Varun Ponugoti | **Code:** https://github.com/Varun-Ponugoti/UR5-Robot.git | **Simulation:** https://youtu.be/3HzRSJdKg3U

1 INTRODUCTION

The main goal of this project is to understand and develop an interactive stick figure simulation for a 6-DOF UR5 robot arm that integrates forward kinematics, inverse kinematics, and trajectory planning. I first derived the forward kinematics of the UR5 using standard DH parameters following the general formulation described in Kebria et al. (1) which compute the pose of the end-effector for any set of joint angles.

Once the forward kinematics were clear the next step was to do the opposite: find the joint angles required to reach a desired end-effector pose. For this, I implemented Newton Raphson IK solver from scratch following iterative numerical ideas widely applied in UR5 IK formulations such as those explained in Villalobos et al. (2). Then, I assumed my robot to be at its home position with all the joint angles as 0 degrees. To make the system practical, I added singularity detection using three methods: determinant checking, condition number analysis, and Yoshikawa's manipulability measure. The system continuously monitors the robot configuration and can detect when the robot approaches or enters a singular configuration where it loses degrees of freedom.

For trajectory planning, I implemented three path types (line, square, circle) which are constrained to 2D planes perpendicular to a selected axis. Each trajectory is generated as a sequence of waypoints with the IK solver computing joint angles for each step and forward kinematics moving the robot to that position. If a singularity is detected at any waypoint the robot's movement immediately stops and displays a warning. This report explains the steps I took, the methods I used, and the results I obtained, along with a short video showing the animation of the UR5 following the trajectory.

2 FORWARD KINEMATICS

In this section, I derive the homogeneous transformation from the base frame to the end effector using the Denavit–Hartenberg (DH) convention and the published link parameters of the UR5. By systematically assigning coordinate frames to each joint and computing the corresponding transformation matrices, I obtained the final T_{06} matrix that expresses the end-effector position and orientation.

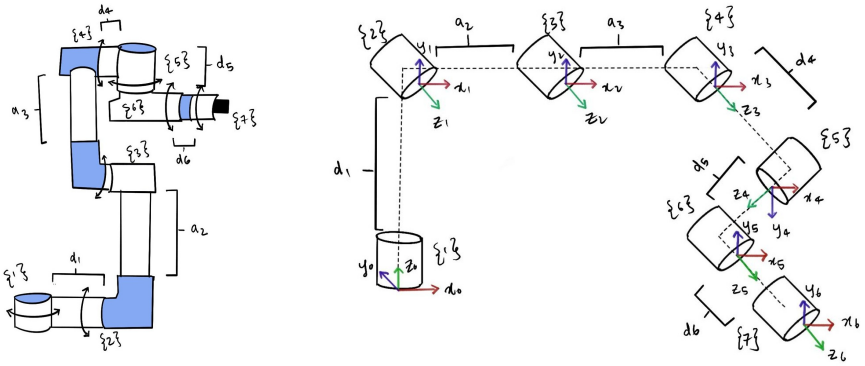

Figure 1. Schematic diagrams of UR5 Robot in home position to understand the coordinate frames

Transformation (T_{ij})	i	d_i	a_i	α_i	θ_i
T_{01}	1	$d_1 = 89.2$	0	$\pi/2$	θ_1
T_{12}	2	0	$a_2 = 425$	0	θ_2
T_{23}	3	0	$a_3 = 392$	0	θ_3
T_{34}	4	$d_4 = 109.3$	0	$\pi/2$	θ_4
T_{45}	5	$d_5 = 94.75$	0	$-\pi/2$	θ_5
T_{56}	6	$d_6 = 82.5$	0	0	θ_6

Table 1. DH parameters of the UR5 Robot

The general DH transformation from frame i to j is given by the standard translation–rotation sequence:

$$T_{ij} = R_z(\theta_i)\, T_z(d_i)\, T_x(a_i)\, R_x(\alpha_i) = \begin{bmatrix} \cos\theta_i & -\sin\theta_i \cos\alpha_i & \sin\theta_i \sin\alpha_i & a_i \cos\theta_i \\ \sin\theta_i & \cos\theta_i \cos\alpha_i & -\cos\theta_i \sin\alpha_i & a_i \sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

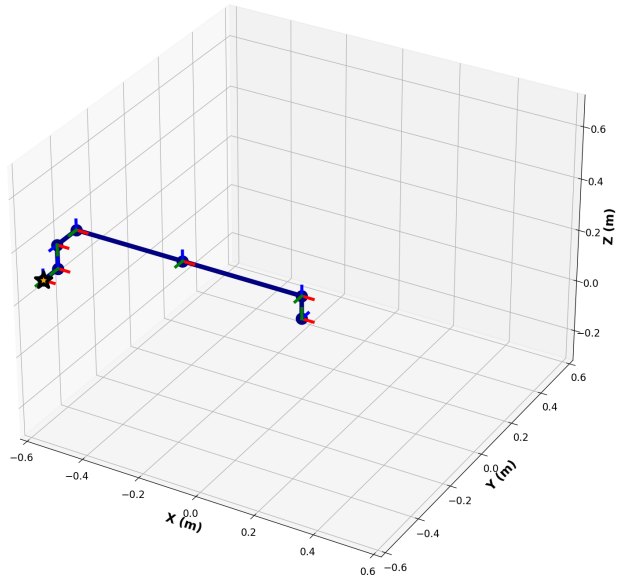

Figure 2. Home Position

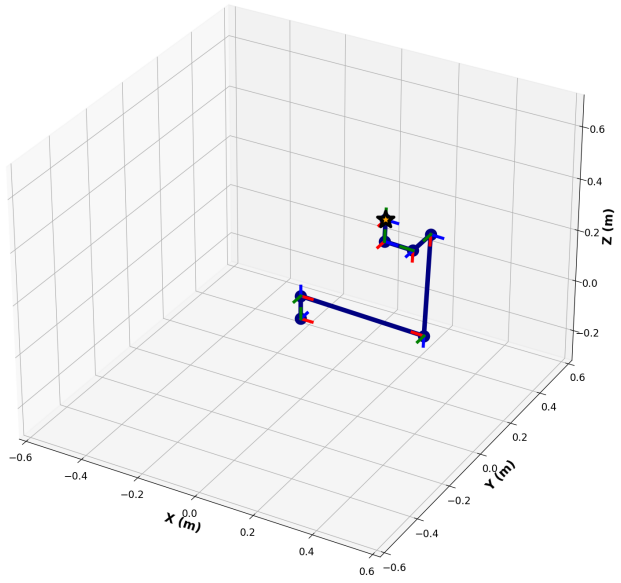

Figure 3. Arbitrary Position

At the home position, the joint angles are:

$$\theta_1 = \theta_2 = \theta_3 = \theta_4 = \theta_5 = \theta_6 = 0$$

Using these values, the forward kinematics of the UR5 are obtained by:

$$T_{06} = T_{01}\,T_{12}\,T_{23}\,T_{34}\,T_{45}\,T_{56}$$

Forward kinematics matrix:

$$T_{06} = \begin{bmatrix} 1 & 0 & 0 & -0.817 \\ 0 & 0 & -1 & -0.192 \\ 0 & 1 & 0 & -0.006 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

At an arbitrary position, the joint angles are:

$$\theta_1 = 0^\circ, \quad \theta_2 = 180^\circ, \quad \theta_3 = 90^\circ, \quad \theta_4 = 0^\circ, \quad \theta_5 = 90^\circ, \quad \theta_6 = 0^\circ$$

Using these values, the forward kinematics of the UR5 are obtained by:

$$T_{06} = T_{01}\,T_{12}\,T_{23}\,T_{34}\,T_{45}\,T_{56}$$

Forward kinematics matrix:

$$T_{06} = \begin{bmatrix} 0 & 1 & 0 & 0.338 \\ -1 & 0 & 0 & -0.105 \\ 0 & 0 & 1 & 0.549 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3 INVERSE KINEMATICS

In this section, I used the Newton Raphson method to solve the inverse kinematics for the UR5 Robot. The Newton-Raphson method updates the joint angles iteratively using the manipulator Jacobian.

$$\boldsymbol{\theta}_{\text{next}} = \boldsymbol{\theta}_{\text{current}} - J^{-1} \cdot \text{error}$$

where J is the Jacobian matrix representing the sensitivity of the end-effector position to changes in joint angles, and error is the difference between the current and desired end-effector positions. The iteration continues until the error falls below a defined tolerance.

3.1 Space Jacobian

For an n-DOF manipulator, the space Jacobian is a $6 \times n$ matrix in which each column represents a single joint and is determined by the joint's axis of rotation or translation and its relative position to the end-effector.

In the UR5 robot, the Jacobian is a 6×6 matrix. At the home position, the space Jacobian can be directly represented by the screw axes of each joint expressed in the base frame:

$$J_s = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & -1 & -1 & 0 & -1 \\ 1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 89.2 & 89.2 & 89.2 & 109.3 & -5.55 \\ 0 & 0 & 0 & 0 & 817 & 0 \\ 0 & 0 & -425 & -817 & 0 & -817 \end{bmatrix}$$

For configurations away from the home position, the joint axes are no longer aligned with their initial directions due to joint rotations. To account for this, the space Jacobian at an arbitrary configuration is computed using the adjoint of the cumulative transformations of all preceding joints:

$$J_s(\boldsymbol{\theta}) = \begin{bmatrix} S_1 & \text{Ad}_{T_1} S_2 & \text{Ad}_{T_1 T_2} S_3 & \dots & \text{Ad}_{T_1 T_2 \dots T_5} S_6 \end{bmatrix}$$

3.2 Damping Factor

When using the Newton-Raphson method for inverse kinematics, the update step involves inverting the Jacobian or computing a pseudo-inverse. Near singular configurations, the Jacobian does very large joint angle updates that can destabilize the solution or cause unrealistic motions. Therefore, a damping factor is introduced using the damped least squares (DLS) method to act as a stabilizer:

$$\Delta\boldsymbol{\theta} = J^T \left(JJ^T + \lambda^2 I\right)^{-1} \text{error}$$

where:

λ is the damping factor, I is the identity matrix

When the robot is far from singularities, the effect of λ is minimal and the solution behaves like a standard Newton-Raphson update. However, near singularities, it prevents excessive joint movements by reducing the effect of very small singular values in the Jacobian for smooth and stable convergence.

3.3 Approach and Results Analysis

The Newton Raphson method with a damped least squares formulation is used to update joint angles iteratively based on the space Jacobian. Singularities are detected and handled during the computation to ensure stability while joint limits are enforced to maintain feasible solutions.

Algorithm 1 Inverse Kinematics using Newton Raphson Method

Require: Target end-effector position p_{target}, home joint angles θ_{home}, tolerance tol, maximum iterations max_iter, damping factor λ

Ensure: Joint angles θ, convergence status, iteration count, singularity status

Initialize $\theta = \theta_{\text{home}}$

for iteration = 1 to max_iter **do**

Compute current end-effector position p_{current} using forward kinematics

Compute error: error $= p_{\text{target}} - p_{\text{current}}$

if $\|\text{error}\| <$ tol **then**

Check singularity (manipulability, condition number)

return θ, success = True, iteration, singularity status

end if

Compute space Jacobian J:

if at home position **then**

Use screw axes as columns

else

Transform screw axes using adjoint of preceding joint transforms

end if

Compute damped update:

$$\Delta\theta = J^T(JJ^T + \lambda^2 I)^{-1} \text{error}$$

Update joint angles: $\theta = \theta + \Delta\theta$

Apply joint limits to θ

Check singularity (manipulability, condition number)

end for

return θ, success = False, max_iter, singularity status

In the home configuration, all the joint angles θ_1 to θ_6 are zero. This is the starting point for the iterative inverse kinematics solver. When the end-effector is moved to an arbitrary position $(x = 0.6, y = 0.12, z = 0.33)$ the code computes the corresponding joint angles using:

A. **Error Computation** The difference between the target position and the current end-effector position (obtained from forward kinematics) is calculated.

B. **Jacobian Calculation** The space Jacobian that maps joint velocities to end-effector velocities is computed at the current configuration. Later on, the screw axes are transformed using the adjoint method to find the current pose of the robot.

C. **Newton-Raphson Update** The joint angles are updated iteratively using the damped least squares formula to adjust the joint angles for reducing the position error while maintaining numerical stability

$$\Delta\boldsymbol{\theta} = J^T \left(JJ^T + \lambda^2 I\right)^{-1} \cdot \text{error}$$

D. **Iteration and Convergence** The steps are iterated until the error falls below a specified tolerance.

E. **Resulting Joint Angles** After convergence, the solver computes the joint angles required to achieve the target end effector position:

$$\theta_1 = -307.34^\circ, \quad \theta_2 = -91.76^\circ, \quad \theta_3 = 71.15^\circ,$$
$$\theta_4 = 258.12^\circ, \quad \theta_5 = 13.37^\circ, \quad \theta_6 = 0^\circ$$

4 TRAJECTORY TRACKING

In this section, I implemented trajectory tracking for the UR5 robot to move the end-effector along predefined paths. The paths I considered are straight line, square, and circle. For each trajectory the motion is divided into small steps and at each step the desired end effector position is given to the inverse kinematics solver. The Newton Raphson method is used to compute the joint angles corresponding to each intermediate position. At every step, the solver iteratively adjusts the joint angles to reduce the position error until it reaches the target within a small tolerance. I also monitor for singularities and joint limits during the motion to ensure that the robot moves safely and accurately.

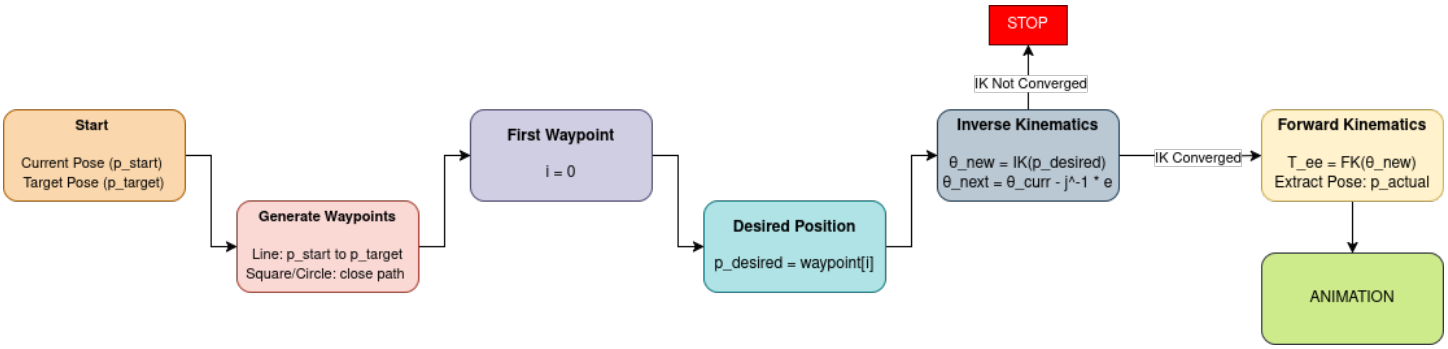

Figure 4. Block diagram of the trajectory tracking pipeline

(i) Line Trajectory

Let $\mathbf{p}_0$ be the initial end effector position and $\mathbf{p}_f$ the final desired position. A discrete set of N intermediate points is generated using linear interpolation:

$$\mathbf{p}_i = \mathbf{p}_0 + \frac{i}{N}(\mathbf{p}_f - \mathbf{p}_0), \qquad i = 1, 2, \ldots, N$$

Each interpolated point $\mathbf{p}_i$ is passed to the inverse kinematics solver. Newton-Raphson IK computes the joint update:

$$\Delta\boldsymbol{\theta} = J^{\dagger}(\mathbf{x}_d - \mathbf{x}_c)$$

where $J^{\dagger}$ is the damped pseudoinverse of the Jacobian.

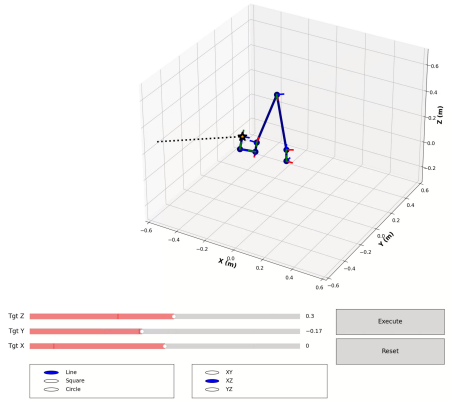

Figure 5. Line Trajectory

(ii) Square Trajectory

A square trajectory is defined by four corner points in the XY, YZ, or XZ plane generated using fixed offsets from the current end effector pose.
The path consists of four linear segments:

$$\mathbf{p}_0 \to \mathbf{p}_1, \quad \mathbf{p}_1 \to \mathbf{p}_2, \quad \mathbf{p}_2 \to \mathbf{p}_3, \quad \mathbf{p}_3 \to \mathbf{p}_0$$

Each segment is separated into N linear interpolation points.
For each point, IK is solved independently:

$$\boldsymbol{\theta}_{i+1} = \boldsymbol{\theta}_i + J^{\dagger}(\mathbf{x}_d - \mathbf{x}_c)$$

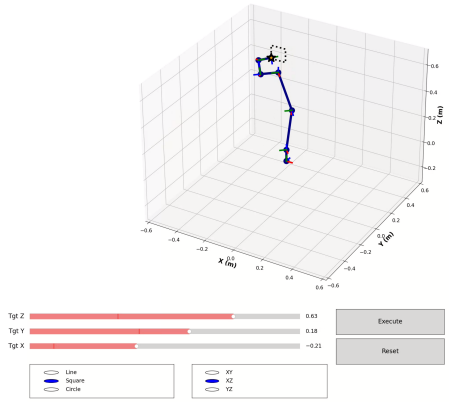

Figure 6. Square Trajectory

(iii) **Circular Trajectory**

Circular motion is parameterized by angle ϕ. For radius r in the XY plane:

$$\mathbf{p}(\phi) = \begin{bmatrix} x_0 + r\cos\phi \\ y_0 + r\sin\phi \\ z_0 \end{bmatrix}$$

The parameter ϕ is sampled uniformly from 0 to 2π.
IK update uses damped least squares to improve stability in low manipulability regions:

$$\Delta\boldsymbol{\theta} = (J^T J + \lambda^2 I)^{-1} J^T (\mathbf{x}_d - \mathbf{x}_c)$$

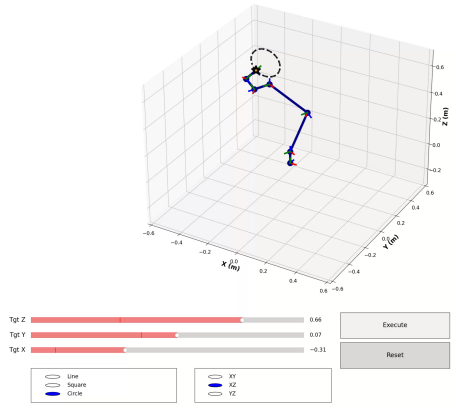

Figure 7. Circular Trajectory

5 CONCLUSION

This project implements a complete kinematic pipeline for the UR5 robot: forward kinematics, iterative inverse kinematics using the Newton Raphson method, and trajectory tracking. By using the damped pseudoinverse and adjoint based space Jacobian updates the IK solver remains stable and effective across a wide range of end effector targets. The robot successfully tracks linear, square, and circular trajectories through stepwise Cartesian interpolation and continuous joint-space updates.

REFERENCES

[1] Parham M Kebria, Saba Al-Wais, Hamid Abdi, and Saeid Nahavandi. Kinematic and dynamic modelling of ur5 manipulator. In *2016 IEEE international conference on systems, man, and cybernetics (SMC)*, pages 004229–004234. IEEE, 2016.

[2] Jessica Villalobos, Irma Y Sanchez, and Fernando Martell. Singularity analysis and complete methods to compute the inverse kinematics for a 6-dof ur/tm-type robot. *Robotics*, 11(6):137, 2022.